R1.02 — Développement d'interfaces web : Architecture Client-Serveur et Protocole HTTP

Thème principal : Architecture Web, Protocole HTTP, Méthodes GET/POST et Codes de Statut

Synthèse exécutive : Ce cours pose les fondements indispensables du Web en expliquant l'architecture client-serveur asymétrique et le fonctionnement textuel du protocole HTTP. Il détaille la nature 'stateless' (sans état) de HTTP, la structure rigoureuse des requêtes et des réponses (lignes de commande/statut, headers, corps), la distinction critique entre les méthodes GET et POST, ainsi que la classification des codes de statut (2xx, 3xx, 4xx, 5xx) pour diagnostiquer et déboguer les applications web.

L'Architecture Client-Serveur et le Principe Stateless

L'architecture client-serveur est le modèle de communication standard du Web, établissant une distinction stricte et asymétrique des rôles. Le client (ex: Firefox, Chrome) prend toujours l'initiative en envoyant un message structuré (la requête HTTP) à travers le réseau. Le serveur reste dans un état d'écoute passive, traite la demande (en exécutant éventuellement des scripts ou en interrogeant une base de données), puis renvoie une réponse. Une caractéristique fondamentale du protocole HTTP est d'être 'stateless' (sans état) : chaque transaction requête-réponse est totalement isolée et autonome. Le serveur oublie instantanément le client dès que la réponse est transmise. Si cette conception simplifie la scalabilité, elle nécessite l'emploi de cookies et de sessions pour maintenir des contextes utilisateurs (comme un panier d'achat).

- Client actif : Navigateur web qui initie la communication en formulant une requête.
- Serveur passif : Logiciel distant (Apache, Nginx) qui écoute sur un port (80/443) et attend les requêtes.
- Protocole Stateless (Sans état) : Absence de mémoire des requêtes précédentes par le protocole HTTP.
- Mécanismes de contournement : Utilisation de cookies et de sessions pour simuler un état persistant.

Cas concret enseignant : *Le professeur explique que lorsqu'un étudiant se connecte sur le site de l'IUT de Bayonne, son navigateur n'envoie pas une seule requête unique, mais des dizaines de requêtes HTTP simultanées pour récupérer le fichier HTML de base, puis chaque image, feuille de style CSS et script JavaScript associé. Il compare également HTTP à un guichetier amnésique qui traite chaque visiteur comme un inconnu, exigeant un badge (cookie de session) à chaque interaction.*

Structure des Messages HTTP et Méthodes (GET vs POST)

Un message HTTP est purement textuel et structuré de manière rigoureuse. La requête comprend une ligne de requête (méthode, URI, version), des en-têtes (headers) fournissant des métadonnées essentielles comme le domaine cible via 'Host' ou le type de client via 'User-Agent', une ligne vide obligatoire, et un corps (body) optionnel. La réponse du serveur adopte une structure similaire mais commence par une ligne de statut. Le choix de la méthode est primordial : GET transmet les données dans l'URL avec des limites de taille, tandis que POST encapsule les données dans le corps de la requête. Attention, POST n'est pas sécurisé par défaut sans chiffrement HTTPS.

- Ligne de requête/statut : Première ligne contenant méthode/URI/version pour la requête, ou code de statut pour la réponse.
- En-têtes (Headers) : Métadonnées clés-valeurs (Host, User-Agent, Content-Type) décrivant le contexte.
- Méthode GET : Conçue pour la consultation, paramètres visibles dans l'URL, sûre et idempotente.
- Méthode POST : Conçue pour modifier l'état du serveur, paramètres encapsulés de manière invisible dans le corps (body).

Cas concret enseignant : *Le professeur insiste sur le fait que le protocole HTTP est purement textuel et en clair par défaut. Utiliser la méthode GET pour transmettre un mot de passe constitue une faille de sécurité majeure, car les données apparaissent en clair dans l'historique du navigateur ou à la vue d'une personne se trouvant derrière l'utilisateur (shoulder surfing).*

La Classification des Codes de Statut HTTP

Chaque réponse HTTP commence par un code numérique à trois chiffres permettant de classifier immédiatement le résultat. Il est crucial pour un développeur de distinguer les erreurs de la famille 4xx, qui incombent au client (mauvaise URL, syntaxe incorrecte), des erreurs de la famille 5xx, qui révèlent un dysfonctionnement interne de l'infrastructure ou du code serveur (script planté, base de données inaccessible), nécessitant une intervention technique directe.

- Codes 2xx (Succès) : La requête a été reçue et traitée avec succès (ex: 200 OK).
- Codes 3xx (Redirection) : Actions supplémentaires requises par le client (ex: 301 Moved Permanently).
- Codes 4xx (Erreur Client) : Anomalie dans la requête du client, URL erronée (ex: 404 Not Found).
- Codes 5xx (Erreur Serveur) : Défaillance interne du serveur malgré une requête valide (ex: 500 Internal Server Error).

Cas concret enseignant : *Le professeur raconte qu'à ses débuts, il avait configuré par erreur une base de données qui plantait à chaque soumission de formulaire, renvoyant systématiquement une erreur 500. Les utilisateurs paniquaient en pensant que le site était piraté alors qu'il s'agissait simplement d'un point-virgule manquant dans son script PHP.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
Réponse du professeur : Avec GET, les informations de connexion (identifiant et mot de passe) s'afficheraient directement dans la barre d'adresse de l'URL, ce qui est extrêmement dangereux (historique, logs, captures d'écran). Avec POST, ces données sont envoyées de manière masquée dans le corps (body) de la requête HTTP. Toutefois, l'enseignant rappelle que POST seul ne suffit pas : sans HTTPS, le trafic reste en clair et peut être intercepté sur le réseau.

■ Exercice & Démonstration d'Application

Énoncé : Analyser la requête HTTP brute suivante et identifier la méthode utilisée, la ressource demandée, l'hôte cible, et expliquer ce que le serveur va renvoyer si la ressource n'existe pas : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

Résolution pas à pas :
1. Analyse de la première ligne : 'GET /images/logo.png HTTP/1.1'. Le premier mot 'GET' est la méthode HTTP utilisée pour récupérer une ressource sans modifier l'état du serveur. L'URI cible est '/images/logo.png' et la version du protocole est 'HTTP/1.1'. 2. Identification de l'hôte : L'en-tête 'Host: iutbayonne.univ-pau.fr' spécifie le domaine cible, indispensable pour le virtual hosting. 3. Analyse de l'agent utilisateur : 'User-Agent: Mozilla/5.0' identifie le navigateur client. 4. Scénario d'absence de ressource : Si le fichier 'logo.png' n'existe pas dans le répertoire du serveur web, celui-ci ne peut pas honorer la demande et renvoie un code de statut de la classe 4xx, précisément le code '404 Not Found'.

Résultat : Méthode : GET, Ressource : '/images/logo.png', Hôte cible : 'iutbayonne.univ-pau.fr', Code de retour en cas d'absence : 404 Not Found (Erreur Client).

■ Auto-Évaluation Flash d'Examen

Q : Qu'est-ce qu'un protocole 'stateless' comme HTTP ?

R : C'est un protocole sans état où chaque requête-réponse est indépendante. Le serveur ne conserve aucune mémoire des requêtes précédentes d'un même client.

Q : Quelle est la différence majeure entre les codes d'erreur de la série 4xx et ceux de la série 5xx ?

R : Les erreurs 4xx indiquent un problème venant du client (URL erronée, syntaxe incorrecte), tandis que les erreurs 5xx signalent une défaillance interne du serveur (bug de code, base de données injoignable).

Q : Pourquoi n'utilise-t-on jamais la méthode GET pour transmettre un mot de passe ?

R : Parce que GET transmet les paramètres directement dans l'URL, les exposant dans la barre d'adresse, l'historique du navigateur et les journaux (logs) du serveur.